PROVEIT RESEARCH CONTRIBUTION

K215 proof simplification

Semantic contracts and a shared implication derivation

Prepared for Vladimir Reshetnikov

21 October 2026

Reviewed repository revision

`6f0d42d368506bc29c70ace1c992ed88b2651346`

VladimirReshetnikov/ProveIt, with the current phase-nine proof organization and the subsequent condensed-detachment changes.

Abstract

The existing K215 correctness development already contains substantial proof compression. The next useful change is to make the generic simulation depend on precise semantic contracts, allowing the existing symbolic checker and shared decoder/primitive proofs to supply the same evidence. We supply a checked Lean implementation of this interface while preserving the original theorem signatures, and additive prefix/suffix composition modules. A concrete decrement site illustrates the composition for arbitrary register values and gaps. Separately, we port the maintained truth-free 58-step implication derivation to Rocq, generate both languages from the same derivation, and instantiate the abstract result at the existing numerical codes. The Rocq core and numerical adapter together use 15,638 bytes, against 22,673 bytes in the maintained numerical replay, a reduction of 31.0%. The unchanged downstream calculus equivalence is checked as well. This packet provides component proofs, compatibility clients, source-bound verification records, and an adoption plan. It does not claim a rebuilt full K215 endpoint after these patches or an already measured reduction in the full site-checking workload.

Reading guide. Sections 1–2 establish the current baseline and the contribution boundary. Sections 3–5 give the simulation and decoder arguments. Sections 6–7 describe the shared implication proof. Sections 8–10 give measurements, validation, and integration. Section 11 proposes further research questions.

Contents

1 Recommendation and scope **2**
1.1 What has and has not been completed . . . 2

2 The exact baseline and theorem to preserve **3**
2.1 The machine and the external endpoints . . . 3
2.2 The existing proof decomposition is already good . . . 3
2.3 Existing results that must not be counted again . . . 4

3 Replace checker-specific hypotheses by semantic contracts **4**
3.1 The current coupling . . . 4
3.2 Positive run contracts . . . 4
3.3 The exact per-edge contract . . . 5
3.4 The generic simulation theorem . . . 6

4 Why physical progress must remain explicit **7**

5 Attach the exact decoder prefix before sharing all primitives **7**
5.1 The maintained entry theorem already has the right quantifiers . . . 7
5.2 A conservative intermediate implementation . . . 8
5.3 Concrete decrement example . . . 8
5.4 Why a shorter continuation key is not yet justified . . . 8

6 One implication derivation, two independent kernels **9**
6.1 Abstract assumptions . . . 9
6.2 Why this is a useful proof boundary . . . 9
6.3 A checkable derivation rather than a trusted translator . . . 10

7 The Rocq numerical adapter and measured saving **10**
7.1 Instantiation at the actual numerical codes . . . 10
7.2 Counts that include the new dependency . . . 11
7.3 Toolchain scope . . . 11

8 Occurrence census and the scale of the next experiment **11**

9 Validation and reproduction **12**
9.1 What the evidence records . . . 12
9.2 Reproduction commands . . . 13
9.3 Compatibility means a stated interface, not every historical helper . . . 13

10 An integration sequence with concrete acceptance gates **14**
10.1 First change: the shared Rocq logical core . . . 14
10.2 Second change: semantic coverage with old producers . . . 14
10.3 Third change: suffix checks for an entire existing block . . . 14
10.4 Fourth change: share families of suffixes . . . 14

11 Further research questions **15**

A Formal reasoning boundary **15**

B Artifact map and provenance **16**

1 Recommendation and scope

The recommended architecture has two reusable mathematical interfaces:

1. A *semantic site interface* between the binary runtime and the register-machine simulation. Its evidence consists of exact, positive runs with the existing path, register, spare-block, overflow, and terminal conditions.
2. A *representation-independent implication interface* between the dominated calculus and its numerical encoding. Its evidence consists of modus ponens, restricted instances of one implication scheme, and elementary closure properties of a relation.

The first interface addresses the largest remaining repeated proof workload. The second provides an immediate, measured reduction in the Rocq calculus proof and aligns the mathematical derivation used by the two proof assistants. Neither requires modifying the 215-state machine.

This is an incremental contribution against a pinned, recent tree. The repository already includes ordinary and normalized certificate symmetry, shared source data, a shared Home scanner, a representation-independent Lean implication core, and exact decoder-entry proofs. Earlier proposal directories also describe semantic site contracts and contain multiple mirror lemmas. Reintroducing those as new ideas would not advance the task. Our contribution is a checked interface implementation, a checked route from decoder prefixes to site contracts, and a numerical Rocq instance of the current shared core. The preceding architecture and derivation retain their original attribution.[2, 5, 8, 11]

1.1 What has and has not been completed

Component	Delivered result	Remaining endpoint work
Lean simulation	Semantic site contracts, checker adapter, generic simulation, original API clients	Rebuild the full endpoint closure when incorporating the replacement
Decoder composition	Additive prefix/suffix bridge and a concrete first-site example	Produce and check suffix evidence for all site occurrences before replacing full-site coverage
Rocq implication	Generated 58-step core, numerical adapter, unchanged downstream calculus client	Wire the producer into the production regeneration manifest and rerun the production endpoint
Finite census	Exact table/source hashes, source-tree checks, occurrence and branch counts	This is an audit of finite inputs, not a kernel correctness theorem

The simulation interface adds source before the large repeated site computations are migrated. Its immediate benefit is a smaller dependency contract, not a claim that the entire codebase has become shorter. The Rocq change does have a directly measured source reduction, counting both newly required files. No global line-count, runtime, proof-object, or minimum-state claim follows from these local results.

2 The exact baseline and theorem to preserve

2.1 The machine and the external endpoints

The pinned dense table has 215 working states, two symbols, and 430 transition rows. Its SHA-256 is

`fdbcef0825886035c3091c8537a69535bc96`
`f9e265f0ae45863543d40ae70090`

where the line break is typographic only. The source contains 4,751 primitive occurrences. These facts are independently recomputed by `audit_snapshot.py` from the included, hash-bound inputs. The table and machine convention are inherited from the maintained K215 candidate.[1]

The Lean endpoint to preserve is

```
KEndpoint215.blank_halts_iff_zfc_refutation :
  BlankHalts machine215 <-> BProv ZFCax_s [] Form.fBot
```

and its nonhalting version is the propositional negation of this equivalence. The corresponding Rocq endpoint is `K215F.CorrectDense215.tm215_halts_iff_ZFC_inconsistent`.[4, 3]

These external proof-assistant statements identify halting with the existence of a finite syntactic refutation of ZFC. They do not assume consistency and do not conclude unconditional nonhalting. The repository separately distinguishes these theorems from a derivation encoded within PA or within ZFC's own proof syntax. The present contribution preserves that distinction: it simplifies components of the external theorem, and does not supply a new internalization theorem. The ordinary mathematical objective that ZFC proves the consistency calibration is therefore not silently conflated with a kernel theorem about an externally defined derivability predicate.[1, 3]

2.2 The existing proof decomposition is already good

Layer	Responsibility	Representative interface
Dense table	Exact renaming between public states and runtime labels	`commutes`, terminal rows
Blank startup	Reach the intended dispatcher frame from blank tape	`boot`, `boot_rel`
Runtime	Source steps correspond to positive binary runs	`KSim.halts_iff`
Source program	Literal source agrees with the enumerator DAG	`KSource215.p215_halts_iff`
Enumeration	A successful enumeration corresponds to a derivable contradiction code	`dagPerm_seed_iff`, `thmDom_iff`
Foundations	Derivable contradiction corresponds to ZFC refutability	Generic endpoint/calibration chain

The final endpoint is already a short composition. Compressing that composition further would mostly hide informative names. The useful targets are repeated work within the runtime and separate realizations of the same implication argument. In particular, the 6,176 branch outcomes remain in the phase-nine runtime proof. Sharing the Home return proof has improved the reported site

compilation, but has not replaced each occurrence's decoder and suffix obligation by a reusable semantic contract.[5, 6]

2.3 Existing results that must not be counted again

The current Lean implication core is a 58-step, truth-free condensed-detachment derivation. Its file has 295 lines and 13,984 bytes; the numerical adapter adds 73 lines and 2,680 bytes. The earlier 689,857-byte replay is an historical baseline, not the source we are replacing in Lean. The maintained Rocq `DomRho8.v` is still the 22,673-byte numerical replay at this snapshot.[8, 9, 10]

The phase-nine site timings total 984.608 seconds, compared with 1,649.340 seconds in historical phase-six records. This comparison is explicitly historical, not a controlled experiment performed here. The phase-nine documentation also records a rejected cache-reuse attempt, followed by a repaired dependency-sensitive build. These details matter when evaluating a new patch.[5]

A historical Rocq normalized-certificate lane has a successful receipt, but its source hashes do not match all the corresponding production files at the pinned revision. The small audit in `evidence/normalized91_status_audit.json` records the mismatch. Its timing and size results may inform the next integration choice; they are not a receipt for the current production source tree. This is a provenance distinction, not an objection to the normalized proof.

3 Replace checker-specific hypotheses by semantic contracts

3.1 The current coupling

The existing `Covered M p` says that every source walk reaches a primitive whose Boolean `checkEdge M Q e` evaluates to `true`. The generic simulation consequently obtains all its evidence by destructing a particular executable checker. That choice is stronger than the mathematical argument needs. A direct primitive theorem, an exact decoder-prefix theorem composed with a suffix theorem, and a successful full symbolic check should all be admissible evidence for the same simulation.

The refactor leaves the old Boolean functions and original `Covered` definition in place. It introduces propositions describing their semantic consequences, proves that the old checks imply those propositions, and proves the generic simulation from the new interface. The old theorem names become compatibility wrappers. Existing callers can continue to supply the same hypotheses.

3.2 Positive run contracts

Let M be the binary machine, Q a consumed instruction word, b a branch assumption, and E a symbolic target configuration. The existing concretization function $\mathrm{conc}(n, v, S)$ interprets a symbolic configuration S at gap parameter n and register/spare list v. Define

$$\begin{aligned}\mathrm{RunToSpec}(M, b, Q, E) \;:\Longleftrightarrow\; &\forall n \in \mathbb{N}\, \forall v,\ \mathrm{Valid}(b, v) \implies \\ &\mathrm{Reach}^{+}_{M}\big(\mathrm{conc}(n, v, \mathrm{start}(Q)), \mathrm{conc}(n, v, E)\big).\end{aligned}$$

Here Reach^{+} is the existing positive-run relation: the physical execution length is at least one. The quantifiers range over arbitrary natural values and legal lists; they are not bounded tests or representative samples.

The symbolic `Valid` predicate includes the register-list length requirement and, for a decrement, the appropriate zero or nonzero branch condition on the physical register. Logical registers are mapped to physical positions by the maintained `physOf`. This relationship must be retained in the interface; matching the control state alone is insufficient.

The halt case has a different useful contract:

$$\begin{aligned}\mathrm{HaltPortSpec}(M,Q) \;:\Longleftrightarrow\; &\forall n\,\forall v,\ |v|\geq 10 \implies\\ &\exists T,\ \mathrm{Reach}_M(\mathrm{conc}(n,v,\mathrm{start}(Q)),T)\wedge \mathrm{state}(T)=230.\end{aligned}$$

This statement does not yet assume that state 230 is terminal. The original premise

$$\forall a\in\{0,1\},\qquad M(230,a)=\mathrm{none}$$

is used where it already belongs: when converting arrival at the halt port into actual halting. This preserves the old dependency structure and avoids unnecessarily strengthening the checker adapter's assumptions.

3.3 The exact per-edge contract

The new `EdgeSpec M Q e` retains the path bound $|Q|\leq 48$ and uses the same target expressions as `checkEdge`. The following table summarizes the cases; all targets denote complete symbolic configurations.

Edge	Required runs	Preserved data
Increment r	One positive run; $r<9$	Incremented register, next path, marker overflow, spare suffix convention
Decrement r	Two positive runs; $r<9$	Zero branch breaks; nonzero branch decrements and advances
Break k	One positive run	Exact ancestor-sensitive break path and overflow test
Continue k	One positive run	Exact continue path and no-overflow target
Halt	Arrival at the halt port	Full input quantification and terminal-port identity
Internal node	False	A malformed or unresolved source walk cannot masquerade as a halt

For example, the zero and nonzero decrement cases use different successor paths. The nonzero case changes the register box and its growth term. The increment case has the inherited spare-list effect, while decrement and control operations preserve their specified spare data. The refactor keeps all those expressions, including the complete path and tape framing. It does not weaken an exact contract to a mere statement about a single register value.

Theorem 3.1 (Existing-checker adapter)**.** *Assume the existing* `BoxLemmas M`*. If* `checkEdge M Q e = true`*, then* $\mathrm{EdgeSpec}(M,Q,e)$*.*

Proof. Split on the edge. The Boolean conjunctions give the path and register bounds and the required successful symbolic runs. For a nonhalting branch, apply the existing executor soundness theorem with the unchanged branch validity premise. Its positive-progress conclusion yields `RunToSpec`. For halt, the executor produces a reachable symbolic configuration whose control is 230; concretization preserves

that control, yielding `HaltPortSpec`. The internal-node branch is excluded because its Boolean check is false. The implementation is `edgeSpec_of_checkEdge`; it introduces no new executable rule. □

Definition 3.2 (Semantic coverage)**.** SemanticCovered(M, p) means that for every source counter word P there are an edge e and consumed length d such that the actual source walk returns (e, d), and

$$\mathrm{EdgeSpec}\big(M, \mathrm{consume}(P, d), e\big).$$

The consumed word includes the walk's implicit zero padding. An arbitrary instruction suffix cannot replace it. Totality of the walk, the exact consumed length, and rejection of internal-node leaves are all part of the usable evidence.

3.4 The generic simulation theorem

Let $R_p(c, T)$ be the maintained `KSim.Rel`. It relates a clean source configuration c to a dispatcher configuration T, including the source path, legal gap, complete physical register list, and a nonempty spare suffix.

Theorem 3.3 (Semantic simulation)**.** *If* SemanticCovered(M, p)*, then:*

1. *From $R_p(c, T)$ and a source transition $c \longrightarrow_p c'$, one obtains a configuration T' with $R_p(c', T')$ and $T \longrightarrow_M^+ T'$.*
2. *A source configuration with no successor is an actual source halt.*
3. *If the halt-port rows are terminal, an actual source halt related to T implies that M halts from T.*

Consequently, under the terminal-row premise,

$$R_p(c, T) \quad \Longrightarrow \quad \mathrm{Halt}_p(c) \iff \mathrm{Halt}_M(T).$$

Proof. For the first claim, semantic coverage supplies the actual primitive reached by the source walk. Its length bound permits the existing `rel_view` theorem to rewrite the concrete dispatcher as the concretization of `start Q`. Split on the primitive and, for decrement, on the register value. Apply the appropriate positive run contract. The existing register-layout and path-length lemmas reassemble $R_p(c', T')$ at the exact target.

For the second claim, use total coverage to exclude a failed walk. An internal node is impossible by `EdgeSpec`. Every nonhalting primitive has a successor, so the only remaining case is the designated halt edge. For the third claim, the halt contract reaches control 230, and the terminal-row premise makes its next transition undefined.

The forward halting implication is induction on the number of source transitions. Each source step contributes a finite positive binary block; the final source halt supplies the final binary halting run.

For the reverse implication, induct strongly on the length t of a binary run to a terminal configuration. If the source has no successor, the second claim finishes the proof. Otherwise the first claim gives a positive binary block of length $j + 1$. Determinism implies that this block and the given halting run agree on their common prefix. The block cannot continue past a terminal configuration, hence $j + 1 \le t$. The remaining binary halting run has length $t - (j + 1) < t$, so the induction hypothesis applies to the related successor. Prepend the source step. □

This proof is the existing simulation argument factored through its actual premises. The Lean implementation is in the `KSim.Semantic` namespace. The old `sim_step`, `sim_halt`, and `halts_iff`

follow by converting ordinary coverage to semantic coverage. The original `sim_stop` proof is retained separately so that it still requires no box-lemma argument. The compatibility client checks the old argument order and types.

4 Why physical progress must remain explicit

It is tempting to simplify the interface further by replacing positive runs with possibly empty runs. That change is unsound without another progress argument.

Proposition 4.1 (Zero-step collapse)**.** *Forward simulation by possibly empty runs, together with preservation of source halts, does not in general imply reflection of halting.*

Proof. Take a source system with one configuration c and transition $c \to c$. Take a target system with one configuration T and no outgoing transition. Relate c to T. Every source step is simulated by the zero-step identity run from T to T. There is no source halt, so source-halt preservation holds vacuously. Nevertheless the target halts immediately and the source never halts. □

A well-founded rank on source transitions simulated by zero target steps could repair a more general interface. It would add a proof obligation and is unnecessary for the present K215 complete-edge contracts. The current positive runtime supplies exactly the measure needed by the reverse implication.

Subfragments are different. A decoder prefix may take zero steps when its preceding word is empty. The correct rule is

$$T \longrightarrow_M^* U \quad \wedge \quad U \longrightarrow_M^+ V \quad \Longrightarrow \quad T \longrightarrow_M^+ V.$$

Thus a nonnegative decoder clock and a positive primitive/return clock compose safely. Imposing positivity on every internal fragment would complicate proofs without strengthening the complete-edge conclusion.

The actual K215 source has word lengths between 3 and 47, so its concrete decoder prefixes are positive as a consequence of the existing length bound. The additive composition theorem deliberately remains valid for empty prefixes too. It should express the mathematically necessary condition, not a gratuitous property of this one source census.

5 Attach the exact decoder prefix before sharing all primitives

5.1 The maintained entry theorem already has the right quantifiers

For each source leaf word Q, the existing `KDecoderPrefix215.leaf_prefix` provides a split $Q = pre +\!\!+ [b]$, a physical control q, the corresponding primitive identity and routed-entry evidence, followed by

$$\forall n\ \forall v\ \exists k, \qquad |pre| \le k \le 7|pre|$$

and an exact run

$$\mathrm{conc}(n, v, \mathrm{start}(Q)) \longrightarrow_M^k \mathrm{conc}(n, v, \mathrm{leafPort}(pre, b, q)).$$

The symbolic port retains the entire consumed prefix, reversed on the left of the head, its marker, the final primitive-read bit, the gap, header, and register box.[7]

Crucially, the port is chosen *before* universally quantifying n and v. A port selected independently for each concrete tape would not support one reusable symbolic suffix proof. The maintained

`entry_uniform` theorem establishes precisely this uniformity, using uniqueness of the routed control and word split.

5.2 A conservative intermediate implementation

The proposed migration can begin before proving a complete parameterized primitive library:

1. Retain all existing source-leaf and branch obligations.
2. Use the shared decoder theorem to reach the exact `leafPort`.
3. Run the existing sound symbolic executor from that port to the old exact target.
4. Compose the prefix and suffix runs, obtaining `EdgeSpec`.
5. Supply these contracts to `SemanticCovered` and the generic simulation theorem.

This intermediate route replaces repeated decoder execution by theorem application while preserving an occurrence-specific suffix proof. It does not require proving that all occurrences of a primitive have the same full continuation. It is also easier to compare with the accepted baseline: every old branch remains present, but its executor begins later.

The additive `KSimPrefix` module provides checks and soundness lemmas for symbolic execution from an arbitrary supplied input port. Its prefix-to-edge bridge composes a nonnegative exact prefix with a positive suffix. The `KDecoderSuffix215` module specializes this construction to the literal maintained source and its exact routed entry.

The all-site constructor has two explicit remaining premises: the finite source-shape gate `allSites (fun _ _ => true) [] KSource215.t215 = true`, and `CheckedLeafSuffixes`, which requires the appropriate suffix evidence at every relevant source leaf and routed port. Neither global premise is discharged by this packet. The example below proves `EdgeSpec` for the literal word 000 using its exact state-27 entry and both checked decrement outcomes. It does not instantiate the universally quantified all-site premise or migrate all 6,176 branch outcomes.

5.3 Concrete decrement example

The first source leaf is the word 000, the decrement of logical register 8. Its prefix is 00, its final bit is 0, and its primitive-entry control is 27 in the maintained runtime labels. The prefix follows controls $84 \to 95 \to 27$, moving across the two known zeros and preserving the tape.

The example checks the two suffix branches from this exact port: the zero-register branch with its break successor and the nonzero-register branch with its decremented box and next-path successor. Executor soundness quantifies over every legal gap and list satisfying the corresponding branch condition. This is stronger than replaying the case with a few concrete register values.

The proof uses the maintained literal table and the existing box lemmas. It is a demonstration of the actual new producer interface, not a fabricated abstract machine whose premises were made trivial. Its complete formal source and the compiler record are included with the Lean component.

5.4 Why a shorter continuation key is not yet justified

There are only 23 primitive kinds but 4,751 primitive occurrences. Equal primitive kinds do not imply identical full target paths. A break or continue can leave the repeated subtree, so its result depends

on ancestor structure. Similarly, shared physical control states do not necessarily determine even a local decoder step without predecessor-bit information.

The existing decoder census supplies an explicit example: control 69 reading 0 has different local exits when the preceding bit is 1 and when it is 0. It also exhibits repeated syntactic subtrees whose continue operations return to different outer prefixes. These examples already appear in the repository; the present architecture respects them by retaining complete source words and routed-entry evidence.[12]

The eventual shared primitive theorem should be parameterized by a language of full valid role paths, with an exact transformation of the caller's continuation. A key consisting only of the physical state, current bit, or a short suffix is not a substitute for that theorem. A proof of full continuation equivalence is needed before such a quotient can replace occurrence-specific evidence.

6 One implication derivation, two independent kernels

6.1 Abstract assumptions

Let A be any carrier with a binary operation $\Rightarrow$, and let $\preceq$ be a relation satisfying

$$u \preceq u, \tag{1}$$
$$u \preceq x \implies u \preceq (x \Rightarrow y), \tag{2}$$
$$u \preceq y \implies u \preceq (x \Rightarrow y). \tag{3}$$

There is no assumption that $\preceq$ is transitive, antisymmetric, linear, or a numerical ordering. Define

$$\rho_8(a, b, c, u) = ((c \Rightarrow u) \Rightarrow a) \Rightarrow ((a \Rightarrow c) \Rightarrow (b \Rightarrow c)).$$

Let $T \subseteq A$ be closed under modus ponens, and assume only the restricted row:

$$u \preceq a \quad \implies \quad T(\rho_8(a, b, c, u)).$$

Theorem 6.1 (Shared dominated-implication core)**.** *Under these assumptions, the following hold for all parameters:*

$$T(\rho_8(a, b, c, u)),$$
$$T((a \Rightarrow b) \Rightarrow ((b \Rightarrow c) \Rightarrow (a \Rightarrow c))),$$
$$T(a \Rightarrow b) \quad \implies \quad T((x \Rightarrow a) \Rightarrow (x \Rightarrow b)).$$

This theorem and the 58-step derivation are inherited from the current Lean core. The contribution here is a Rocq rendering of that same abstract derivation and its checked numerical instance. The three conclusions correspond to `rho8_all`, `composition`, and `lift_right`.[9, 8]

6.2 Why this is a useful proof boundary

The abstract theorem needs no truth constant, bottom code, formula decoder, injectivity of coding, special equation such as $0 \Rightarrow 0 = 0$, or semantic completeness argument. All information about numerical encoding is confined to a small adapter that verifies (1)–(3). A change to coding or an alternative minimally constrained numerical calculus can therefore reuse the logical derivation.

The adapter may retain a truth field for source compatibility with the existing `DomImp` record. Its presence in that record does not mean the core derivation uses it. Conversely, proving that this particular implication theorem does not need truth does not remove truth from the complete K215 calculus or justify deleting machine instructions. The full calculus still has its other rules and seed obligations.

6.3 A checkable derivation rather than a trusted translator

The neutral `derivation_core.txt` records formulas, their major and minor premises, side conditions, and the three final goals. The existing producer recomputes the unifier for each condensed-detachment step. The new `-core-rocq` mode emits ordinary Rocq lemmas, while the existing `-core` mode emits Lean lemmas.

Each emitted step is an explicitly instantiated application of modus ponens to two preceding results or an instance of the restricted row. Each side condition is a hypothesis or follows by reflexivity and insertion into an implication operand. The kernel does not trust the producer's unification or search code: it type-checks the emitted formulas, instantiated premises, side conditions, and conclusion.

For orientation, the final core steps expose small reusable formulas. The derivation includes

$$T((a \Rightarrow b) \Rightarrow ((x \Rightarrow a) \Rightarrow (x \Rightarrow b)))$$

as its prefixing formula. Applying modus ponens to a supplied $T(a \Rightarrow b)$ gives `lift_right`. Another final lemma is

$$T(((c \Rightarrow u) \Rightarrow a) \Rightarrow ((a \Rightarrow c) \Rightarrow (b \Rightarrow c))),$$

which is the unrestricted row after a renaming of variables. The final composition formula is a separate goal of the same derivation. The complete 58-step proof is delivered as generated source; the report does not replace it by a claim that a search tool found a proof.

The generator regression checks show that the new mode preserves the existing Lean core byte for byte and preserves both earlier numerical renderings. This separates the new output mode from unrequested changes to already checked code.

7 The Rocq numerical adapter and measured saving

7.1 Instantiation at the actual numerical codes

The Rocq adapter uses $A = \mathbb{N}$, the existing numerical implication `wim`, and ordinary $\leq$. The existing pairing bounds imply

$$x \leq \mathrm{wim}(x, y), \qquad y \leq \mathrm{wim}(x, y).$$

Combining these with a given $u \leq x$ or $u \leq y$ proves the two insertion rules. Reflexivity is the ordinary arithmetic theorem. The numerical `rho8` unfolds to the same abstract operation, so the core theorem directly supplies the old `rho8_all` statement for every raw numerical code.

The adapter preserves the `DomImp` record's three fields and the original `rho8_all` interface. It additionally exposes the truth-free numerical theorem `rho8_all_of_restricted` and the two generic consequences. The unchanged downstream `ThmDom.v` compiles against this adapter, and its `thmDom_iff` assumption report is closed. This is the relevant calculus compatibility check; merely compiling an isolated abstract theorem would not test the numerical attachment.

7.2 Counts that include the new dependency

Rocq source being compared	Lines	Bytes
Maintained numerical `DomRho8.v`	1,169	22,673
New abstract core plus numerical adapter	329	15,638
Reduction	840	7,035
Reduction as a percentage	71.9%	31.0%

Counts include comments and blank lines and are measured on UTF-8 source bytes. They exclude the unchanged numerical-code dependencies, compiler libraries, generation scripts, logs, and the paper. Counting only the small adapter and ignoring the abstract core would overstate the saving. The complete downstream chain remains part of the correctness argument even though its files are unchanged.

There is no measured compilation speedup. In the accepted local run, compiling the new core took 0.296 seconds and its adapter took 2.238 seconds, totaling 2.534 seconds, while the baseline numerical proof took 1.881 seconds. These small local observations are not a controlled endpoint benchmark. The demonstrated benefit is the source reduction and common derivation specification.

An earlier 52-step numerical Rocq proposal already produces an 11,190-byte row proof. It is smaller in standalone source than the new generic core plus adapter, uses truth, and proves the row in the numerical setting. It remains a legitimate alternative if minimizing that one file is the overriding objective. The 58-step option is recommended when the goal is to maintain one abstract derivation shared with the existing Lean core and retain composition and lifting as reusable interfaces. There is no claim that 58 is a minimum proof length.[8]

7.3 Toolchain scope

The delivered local check uses Coq 8.18.0, including a matching recursive `coqchk`. The maintained production lane uses Rocq 9.1.0 with its recorded standard-library version. For the local check, small compatibility modules re-export the older standard-library namespaces where the pinned numerical files refer to `Stdlib`. Their sources are included. The underlying numerical definitions are unchanged.

This is a concrete kernel check of the supplied component and its downstream calculus client, not a claim that this turn rebuilt the complete production Rocq binary-machine/ZFC endpoint on its production toolchain. The integration gate must run the production generator, manifest checks, complete endpoint build, assumption reports, and recursive kernel verification on the resulting source tree.

8 Occurrence census and the scale of the next experiment

The supplied audit recomputes the following counts from the exact source input:

Primitive	Occurrences	Branch outcomes
Increment	1,900	1,900
Decrement	1,425	2,850
Break	98	98
Continue	1,327	1,327
Halt	1	1
Total	4,751	6,176

There are 23 distinct primitive kinds and 202 distinct source subtrees. The audit checks that the listed words form a complete prefix-free binary source tree. These are properties of the finite source representation. They do not justify replacing 4,751 occurrence contracts by 23 primitive contracts without a theorem about their contexts.

If a decrement is counted once for each of its two branch obligations, the sum of the preceding-prefix lengths is

$$\sum_{\text{branch occurrences}} (|Q| - 1) = 200{,}893.$$

The inherited decoder theorem bounds its physical prefix clock between $|Q| - 1$ and $7(|Q| - 1)$. Summing those theorem bounds gives

$$200{,}893 \;\le\; \sum k_Q \;\le\; 1{,}406{,}251.$$

This calculation describes the sum of certified physical decoder lengths when counting a prefix for each branch. It is not a measured count of symbolic evaluator steps, Lean kernel reductions, machine runtime of the blank computation, or seconds saved by the new proof. A compiler can share work, and a macro step can represent many physical transitions. We include the census to make the migration's scope exact, not as a speedup estimate.

A second, separately reproducible parser traverses the maintained source nodes, decoder roles, and local block certificates, checking their node and control linkage. Their actual stored prefix witness clocks sum to 165,876 over the 4,751 leaves, or **213,945 when counting a prefix for each branch outcome**; their minimum is 2 and maximum is 55. The corresponding finite representation has 202 source nodes, 234 decoder roles, and 281 local blocks. These are sums of existing block-certificate physical witness clocks, with the same measurement limitations as the bounds above. The parser and exact result are in `evidence/decoder_prefix_census.py` and its JSON output.

The useful experiment is therefore a paired build of one existing site block under identical conditions: current full-site checks versus shared-prefix plus suffix checks, retaining every old target and branch. Measure wall time, peak memory, source bytes, and object bytes separately. Only expand to all blocks after that experiment shows a benefit or resolves an identified proof-maintenance problem.

9 Validation and reproduction

9.1 What the evidence records

The Lean component includes the required project-source closure, the replacement `KSim`, additive prefix modules, original-API clients, and a serial build driver. It uses Lean 4.32.0 and its distributed standard library. No opaque binary proof objects are shipped as a substitute for source. The accepted receipts bind source hashes, compiler commands, return codes, and raw logs.

All 29 included project modules compiled freshly. Their recorded module times sum to 75.38 seconds; this is a component verification time. The seven typed compatibility clients and the literal two-branch example pass. One unchanged dependency retains an unused-`simp` warning; the new components have no compiler warnings. The independent declaration audit finds all 15 preexisting executable definitions and all 22 preexisting explicit theorem statements unchanged after comment and whitespace normalization. The compiled clients additionally test section-inferred parameters, which a textual comparison alone would miss.

The `KSim` replacement itself grows from 356 lines / 16,146 bytes to 460 lines / 20,505 bytes: 104 lines and 4,359 bytes of additional interface and adapter code. The additive prefix modules cost further source. This is an intentional initial cost of enabling proof reuse, and is kept separate from the immediate Rocq source saving.

The semantic simulation theorems report only the standard Lean axioms already admitted by the development. The generic semantic consumer has no new domain-specific axiom. The legacy wrapper uses the same standard-axiom set as its original dependencies. The concrete prefix example must be distinguished from the all-site suffix premise: successfully checking that example does not close the latter.

The Rocq component includes a fresh small numerical closure, the adapter, an explicit compatibility client, generation controls, and recursive kernel checking. Generic and numerical public results, including the unchanged calculus equivalence, print closed assumption reports. The producer is outside the trusted logical boundary; its output is checked by the kernels.

The finite source audit and manifest verifier are ordinary Python programs. Their role is to bind artifacts and reproduce counts. A successful Python audit is not reported as a proof-assistant theorem.

9.2 Reproduction commands

The component READMEs document the exact paths and accepted commands. From the archive root, the intended sequence is:

```
python3 verify.py
python3 audit_snapshot.py --check
python3 rocq/build.py --coqc /path/to/coqc --coqchk /path/to/coqchk \
  --out /absolute/path/to/fresh-rocq-output
python3 lean/build.py --lean /path/to/lean-4.32.0-linux/bin/lean \
  --out /absolute/path/to/fresh-lean-output
pdflatex -halt-on-error report.tex
pdflatex -halt-on-error report.tex
```

Builds should remain serial, following the repository's shared proof-build lock. Fresh output directories prevent stale compiled files from obscuring a dependency mismatch. The accepted logs in the packet describe the actual component checks; they are not extrapolated to the full 192-module maintained endpoint closure.

9.3 Compatibility means a stated interface, not every historical helper

The Lean compatibility client exercises the original named runtime theorem interfaces. The Rocq client exercises the maintained `DomImp` fields, unrestricted row, and downstream calculus equivalence. Internal generated lemma names and obsolete certificate internals are not promised as an API for

unknown clients. Before incorporation, run a repository caller census and retain or deliberately migrate any additional named uses that appear after the pinned revision.

10 An integration sequence with concrete acceptance gates

10.1 First change: the shared Rocq logical core

Add the abstract core, replace the numerical replay by its adapter, and update the generator mode in the production regeneration pipeline. Add the new dependency to the source/manifest accounting. Regenerate in an isolated tree and confirm that the intended changed files are exactly reproduced. Then run the numerical-calculus clients, followed by the full production Rocq endpoint and recursive kernel check. Preserve the original accepted receipts as history rather than overwriting them with component results.

This change is small enough to review independently of the runtime work. Its source saving is established now, and its correctness attachment is local: the unchanged `ThmDom` client is the immediate semantic consumer.

10.2 Second change: semantic coverage with old producers

Apply the `KSim` patch while keeping all existing Boolean site checks. Verify that executable definitions, literal tables, source data, and old theorem types are unchanged. Compile the affected generic closure and original-API clients, then rebuild the full maintained endpoint closure with dependency-sensitive invalidation. At this stage the 6,176 branch computations remain, and the endpoint still obtains coverage through the old producer.

This intermediate state establishes that introducing the interface does not depend on an unproved primitive-sharing claim. It also creates a stable review point: producer replacement can be evaluated separately from consumer refactoring.

10.3 Third change: suffix checks for an entire existing block

For every leaf in one existing block, select the exact uniform routed port, keep the entire source word, and check the old branch-specific suffix target. Use the shared decoder theorem once at the proof level, then the suffix soundness theorem. Compare with the current block under the same toolchain, source ancestry, serial scheduling, and memory limit. Reject a proposed shortcut if it weakens zero-branch, marker, overflow, spare-list, or halt obligations.

If the results justify adoption, expand to all blocks and replace the coverage proof. This is where a reduction in the repeated runtime proof work can first be claimed for the entire machine. A conditional theorem with a remaining all-suffix premise is useful infrastructure, but not that completed milestone.

10.4 Fourth change: share families of suffixes

Only after the occurrence-specific suffix route is validated should the next abstraction group operations. Begin with a well-behaved family such as increment at fixed logical register, parameterized by the full valid source-prefix language and exact next-path transformation. Treat break and continue separately until their ancestor-sensitive behavior has a suitable compositional statement.

For each family, compare the new proof's total dependencies with the occurrence checks it replaces. A theorem that saves a short local proof but imports a much larger semantic development may make maintenance or compilation worse. Shared abstractions should be evaluated at their consumers, not solely at their definition sites.

11 Further research questions

1. **How many suffix theorem families are actually needed?** The finite source has 23 primitive kinds, but control-flow context may split these into more families. Define a certified equivalence on full role paths that preserves the exact continuation transformation, rather than guessing a short key.
2. **Can the prefix/suffix interface be made compositional at the source-DAG level?** A sound interface would need a parameter describing outer continuation boundaries, plus a theorem that source-DAG composition preserves it. Equal subtrees alone are insufficient.
3. **Which proof representation minimizes kernel work?** Compare direct small lemmas, checked certificates, and shared semantic macros using paired builds. Source size, elaboration time, conversion cost, and proof-object size are distinct objectives.
4. **Can one neutral proof specification serve both complete runtime lanes?** The shared 58-step logical derivation demonstrates the pattern on a small component. A runtime specification would also need exact tape semantics, branch validity, normalization transport, and positive progress, independently checked in each assistant.
5. **Can the truth-free implication derivation be shortened further?** The current 58-step proof is a search result, not an optimality theorem. Search with all three public goals together, and measure the complete core plus adapters rather than only the final unrestricted row.
6. **Can the semantic simulation expose a well-founded zero-step extension without burdening K215?** Such an extension may help future machine transformations. It should specialize to the present positive interface without forcing a rank argument on every existing caller.
7. **When should the normalized Rocq source lane replace the production lane?** Reconcile its exact source hashes and regenerate its current dependency closure before reusing its historical performance evidence. This is an integration and validation task, not a missing mathematical mirror lemma.
8. **What is the shortest independently replayable correctness package?** A useful minimization objective includes theorem statements, literal-machine binding, minimal project dependencies, and readable generation evidence. Deleting provenance or replacing checked assumptions by undocumented trust does not count as proof simplification.

A Formal reasoning boundary

The logical flow remains

$$\mathrm{Halt}(M_{215}, \mathrm{blank}) \quad \longleftrightarrow \quad \mathrm{Refutable}(\mathrm{ZFC}),$$

followed by propositional negation. The runtime refactor changes how one supplies a hypothesis of the simulation theorem. The Rocq logical refactor changes the proof of an intermediate implication

theorem. Neither modifies the literal machine, the accepted proof-enumeration language, or the final refutation predicate.

The supplied source assertions are verified at three different levels. Kernel checks verify the formal component theorems. Generation comparisons verify that neutral derivations reproduce the intended text. Finite audits verify bytes, tables, source structure, and counts. Each level is useful, but the latter two cannot replace the first. Similarly, a component kernel check cannot replace the first full endpoint rebuild after a production integration.

B Artifact map and provenance

Path	Purpose
`report.tex`, `report.pdf`	This paper and its complete editable source
`lean/`	Lean semantic interface, prefix/suffix composition, example, minimal sources, patch, driver, and receipts
`rocq/`	Shared 58-step derivation, Rocq generator mode, abstract core, numerical adapter, clients, driver, and receipts
`baseline/`	Pinned source/table inputs and file hashes used in the comparisons
`evidence/`	Reproduced census and architectural/provenance review
`audit_snapshot.py`	Recompute the exact finite source census
`verify.py`, `MANIFEST.json`	Check delivery integrity and the component source bindings
`LICENSE`, `NOTICE.md`, `licenses/`	Project terms and inherited attribution

The machine, calculus, numerical encoding, source compiler, runtime definitions, existing decoder proof, existing implication derivation, and earlier simplification proposals are inherited work. New material in this packet consists of the stated interface/adapter implementations, their integration and component checks, the present analysis and audit, and the report. No discovery-priority claim is made for semantic simulation, prefix composition, or the dominated-implication theorem.

References

[1] ProveIt, *K215 candidate and exact-machine convention*. Pinned revision `6f0d42d368506bc29c70ace1c992ed88b2651346`. candidate-k215/README.md.

[2] ProveIt, *Formal endpoints for the K-family candidates*. k225-formal/README.md.

[3] ProveIt, *K215 verification record*. verification/k215/README.md.

[4] ProveIt, *Exact Lean K215 endpoint*. KEndpoint215.lean.

[5] ProveIt, *Ninth K215 proof-simplification checkpoint*, including the separately scoped timing and dependency receipts. phase9/README.md.

[6] ProveIt, *Decoder sites and register-machine simulation*. KSim.lean.

[7] ProveIt, *Exact zipper entry for the shared decoder*. KDecoderPrefix215.lean, and its literal source binding.

[8] ProveIt, *Rho8 by condensed detachment*, derivations, generator, and status notes. rho8cd/README.md.

[9] ProveIt, *Representation-independent dominated implication*. DominatedImplication.lean.

[10] ProveIt, *Numerical implication adapters and replay*. Lean DomRho8.lean; Rocq DomRho8.v.

[11] ProveIt, *Earlier runtime-context proposals*. further-simplifications/runtime_contexts/README.md. The broad semantic-contract direction predates this packet.

[12] ProveIt, *Context-aware decoder sharing: bounded investigation*. decoder-sharing-research/notes.md. Includes predecessor-bit and outer-continuation counterexamples.